



Digital Design

Chapter 2: Combinational Logic Design

Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Relating Boolean Algebra to Digital Design

Boolean algebra
(mid-1800s)

Boole's intent: formalize human thought

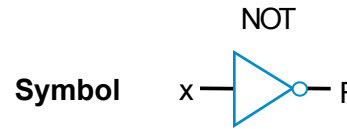
Switches
(1930s)

For telephone switching and other electronic uses

Shannon (1938)

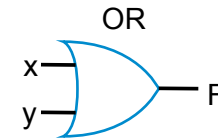
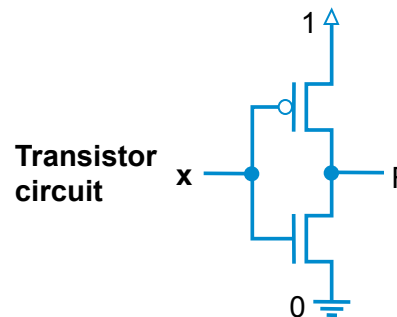
Showed application of Boolean algebra to design of switch-based circuits

Digital design

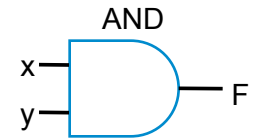
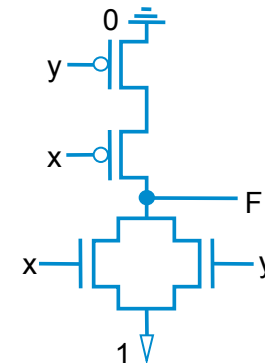


Truth table

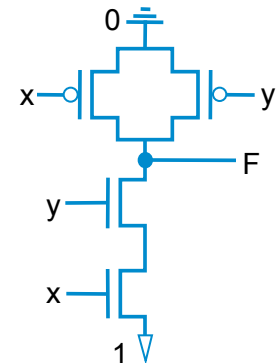
x	F
0	1
1	0



x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

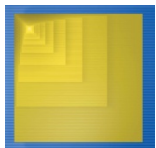


x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

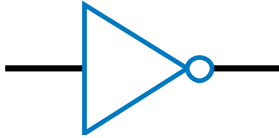


- Implement Boolean operators using transistors
 - Call those implementations **logic gates**.
 - Let's us build circuits by doing math** -- powerful concept

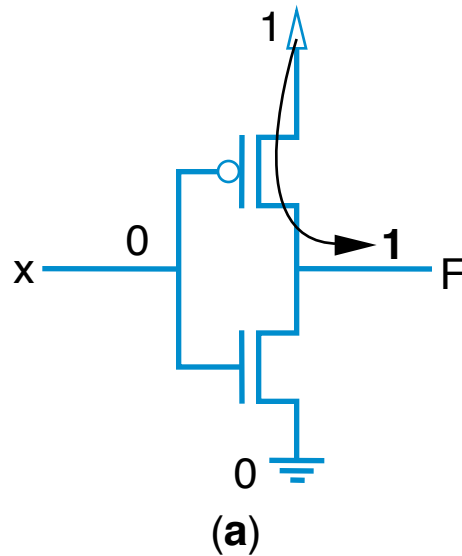
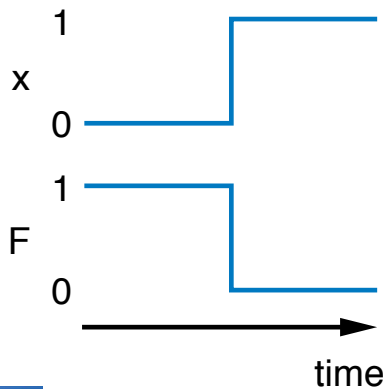
Note: These OR/AND implementations are inefficient; we'll show why, and show better ones later.



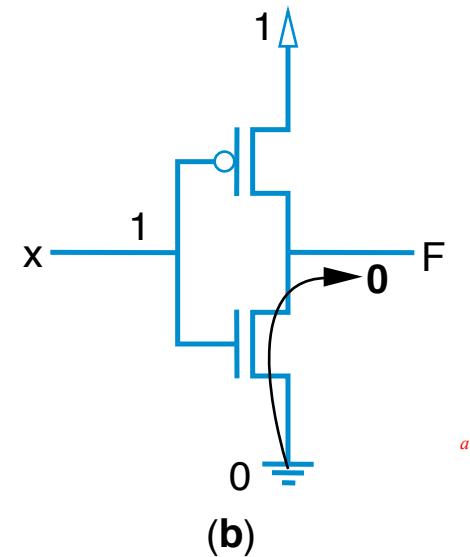
NOT gate



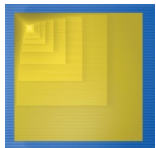
x	F
0	1
1	0



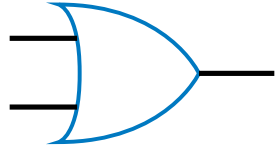
When the input is 0



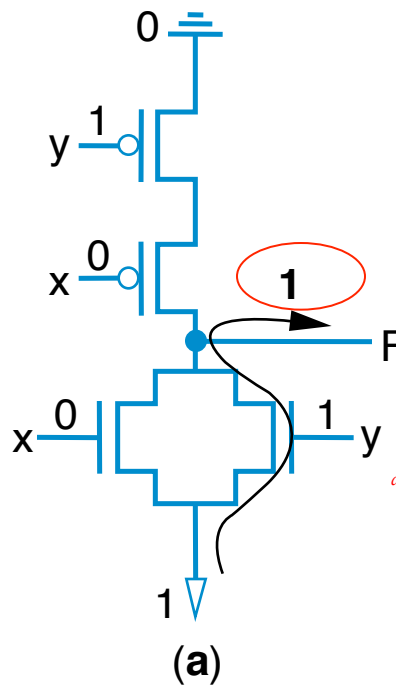
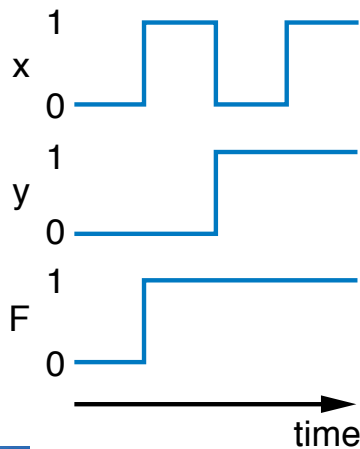
When the input is 1



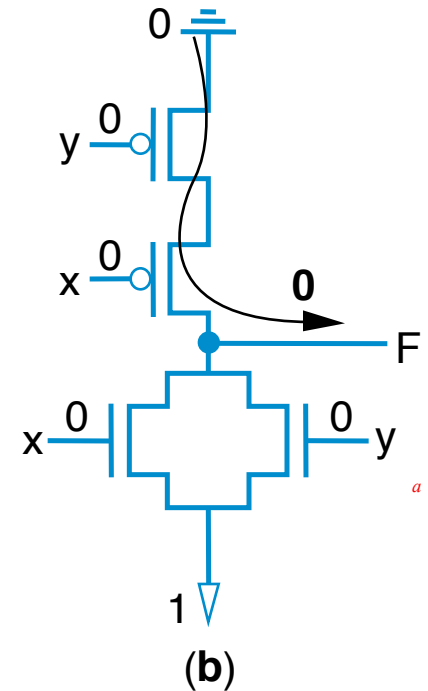
OR gate



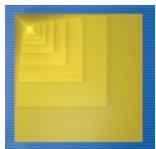
x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



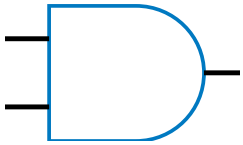
When an input is 1



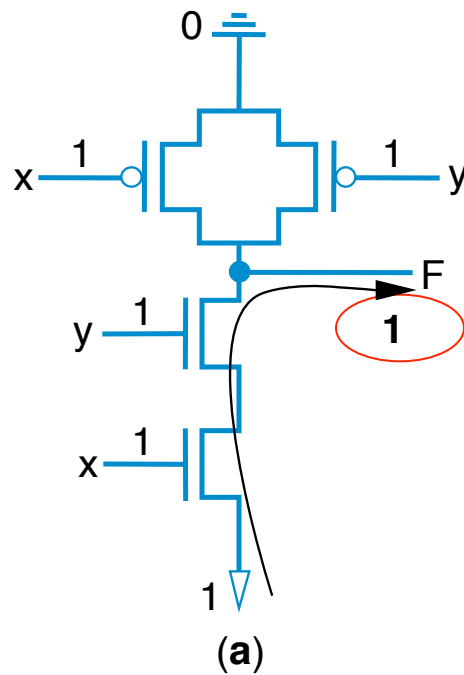
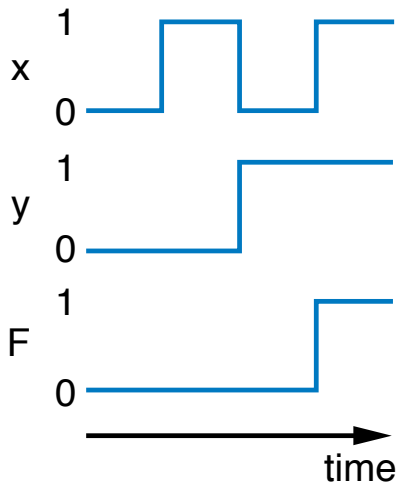
When both inputs are 0



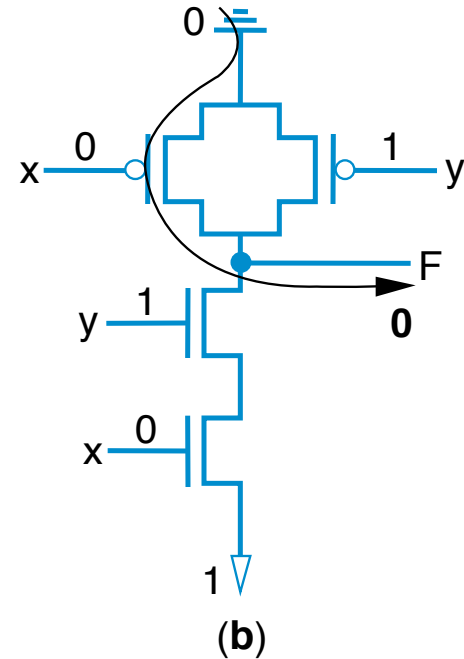
AND gate



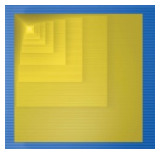
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



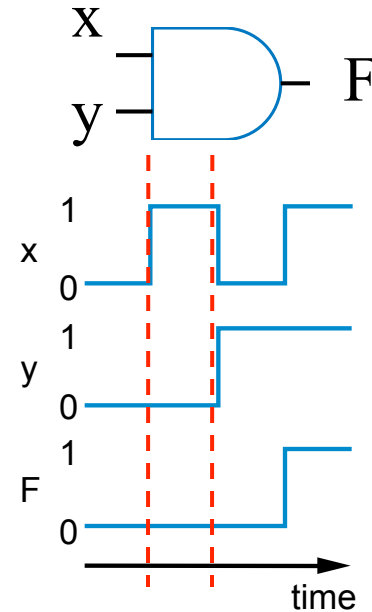
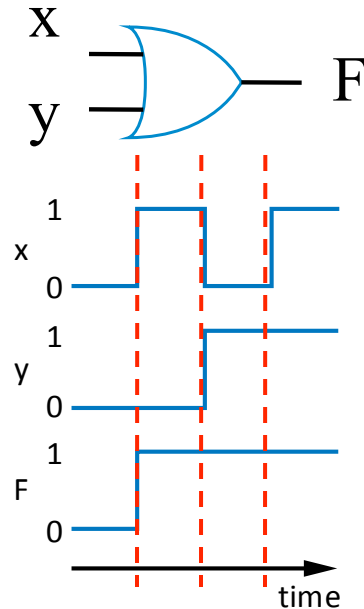
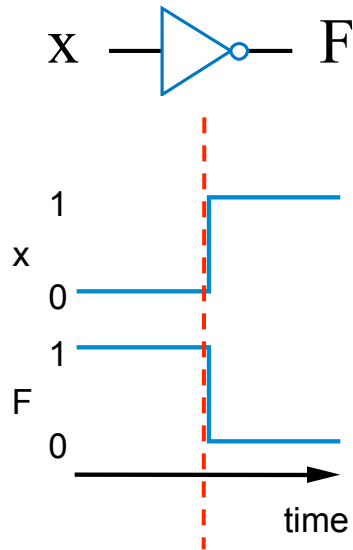
When both inputs are 1



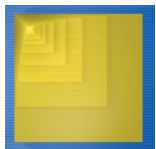
When an input is 0



NOT/OR/AND Logic Gate Timing Diagrams



- Assume zero-delay

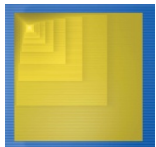


Noise in Computer Systems

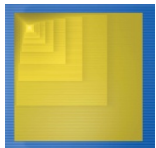
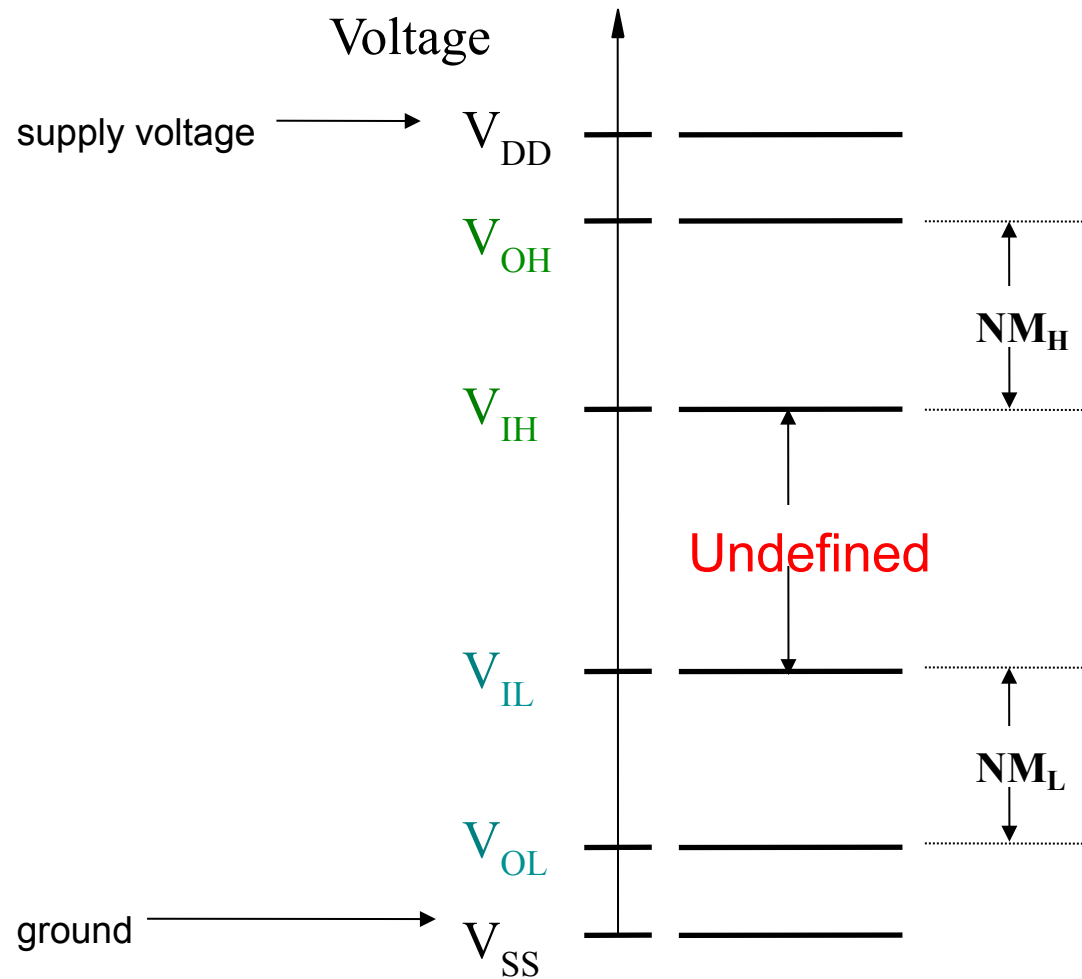
- Computers, like all electronic systems, are affected by noise.
 - noise has various sources (nearby signal changes, thermal vibrations of molecules in semiconductor materials, . . .)
 - in computers, noise can cause binary signals to be misinterpreted
- The **noise margin** is the amount of noise that a system can tolerate and still correctly identify a logic high or low.
- For example, a high of 4.5V at the output of a gate with a noise of 1V brings to 3.5V still high for the input of the next gate.

Output	5v	Input
High	4v	High
	3v	
Undefined	2v	Undefined
	1v	Low
Low	0v	

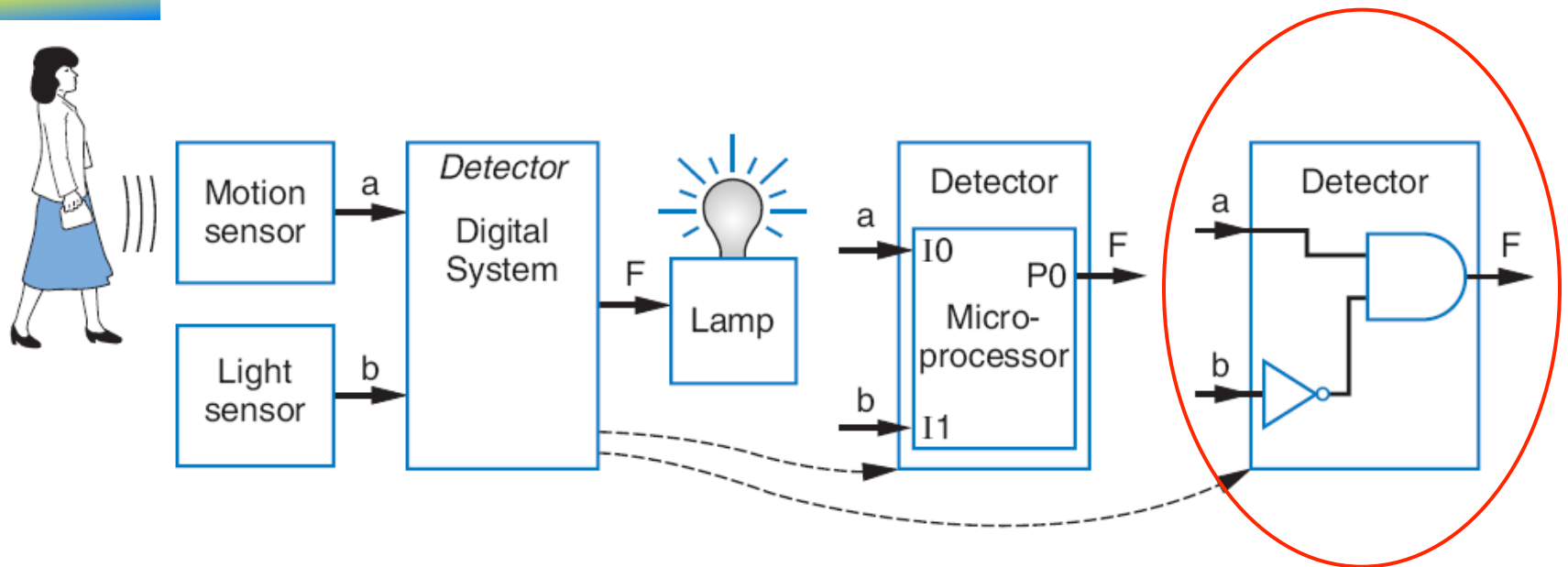
noise margin
1 V



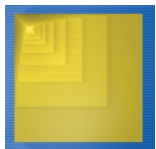
Noise Margin



Building Circuits Using Gates



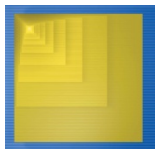
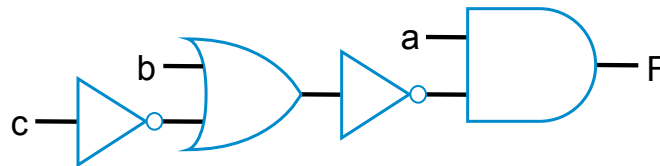
- Recall Chapter 1 motion-in-dark example
 - Turn on lamp ($F=1$) when motion sensed ($a=1$) and no light ($b=0$)
 - $F = a \text{ AND NOT}(b)$
 - Build using logic gates, AND and NOT, as shown
 - We just built our first digital circuit!



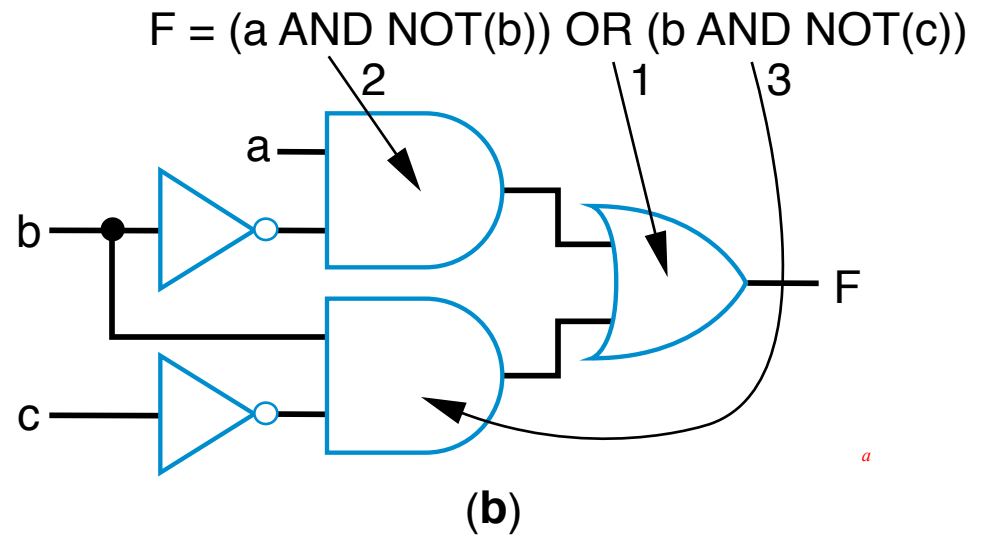
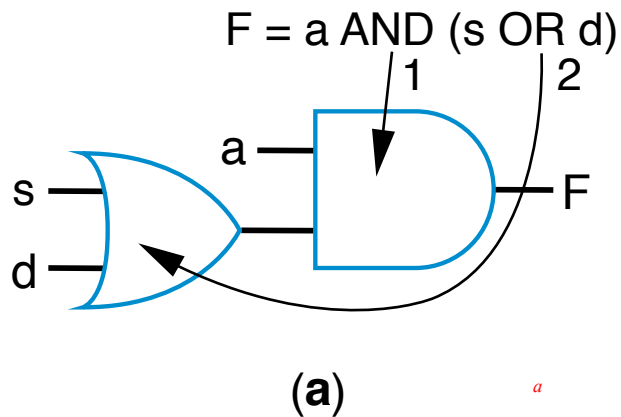
Example: Converting a Boolean Equation to a Circuit of Logic Gates

Start from the output, work back towards the inputs

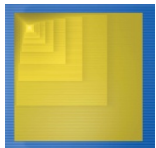
- Q: Convert the following equation to logic gates:
 $F = a \text{ AND NOT}(b \text{ OR NOT}(c))$



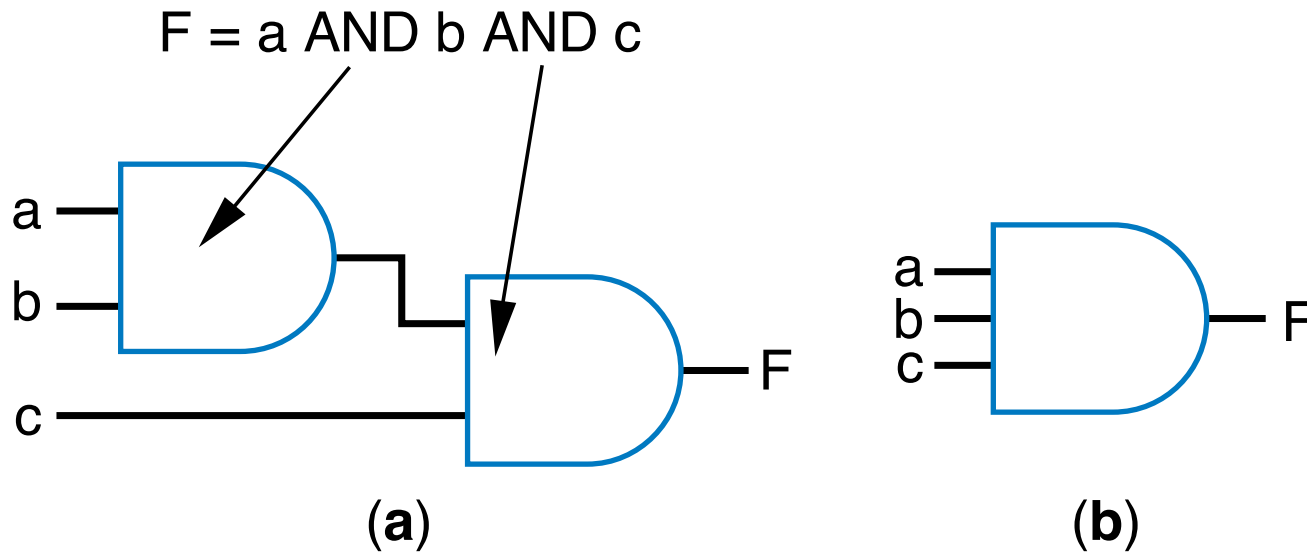
More examples



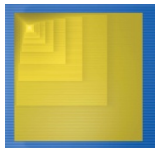
Start from the output, work back towards the inputs



Using gates with more than 2 inputs



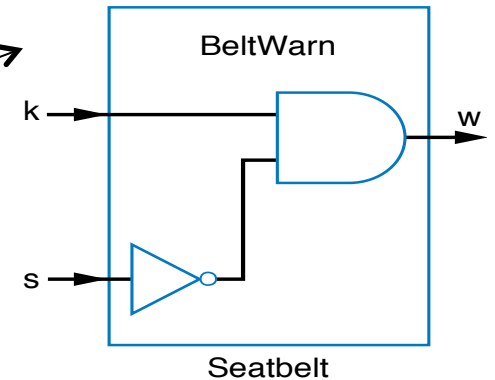
Can think of as $AND(a,b,c)$



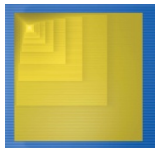
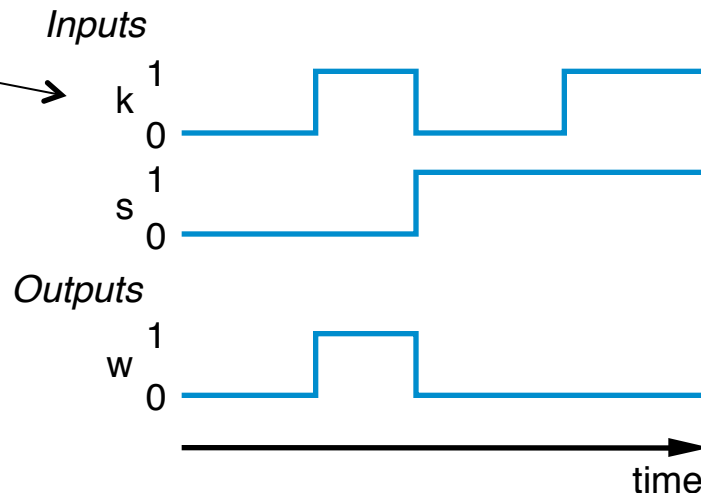
Example: Seat Belt Warning Light System

- Design circuit for warning light
- Sensors
 - $s=1$: seat belt fastened
 - $k=1$: key inserted
- Capture Boolean equation
 - seat belt not fastened, and key inserted
- Convert equation to circuit

$$w = \text{NOT}(s) \text{ AND } k$$



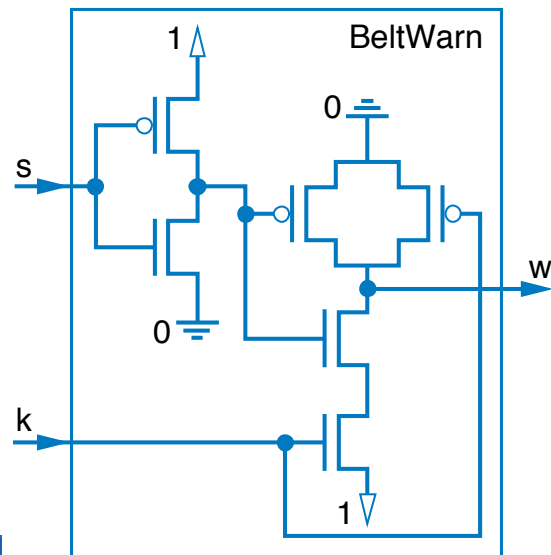
- *Timing diagram* illustrates circuit behavior
 - We set inputs to any values
 - Output set according to circuit



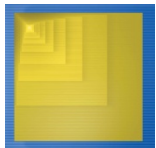
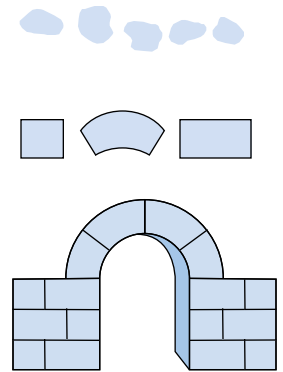
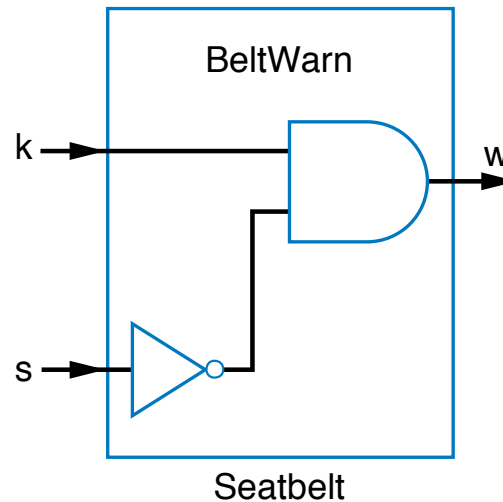
Gates vs. switches

Notice

- Boolean algebra enables easy capture as equation and conversion to circuit
 - How design with switches?
 - Of course, logic gates are built from switches, but we think at level of logic gates, not switches

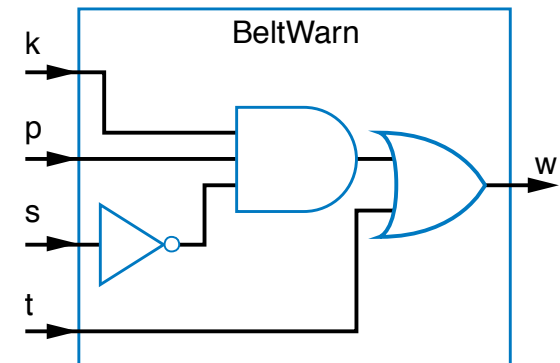
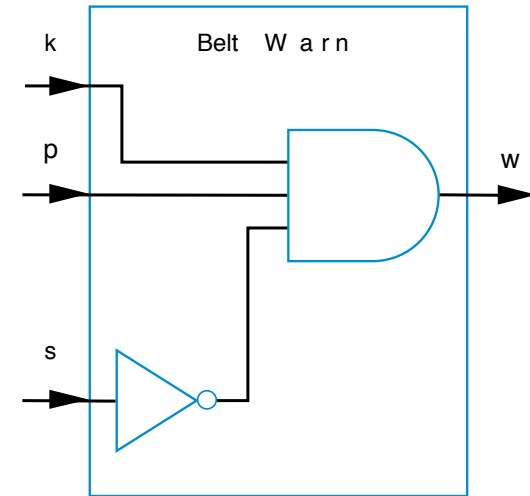


$$w = \text{NOT}(s) \text{ AND } k$$

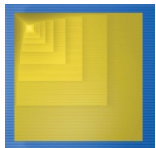
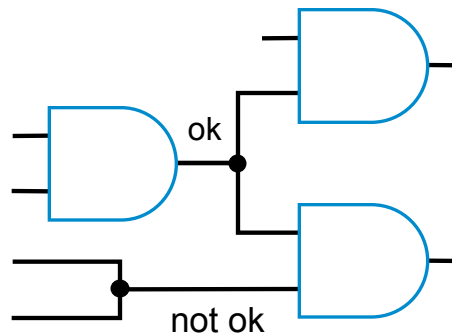


More examples: Seat belt warning light extensions

- Only illuminate warning light if person is in the seat ($p=1$), and seat belt not fastened and key inserted
- $w = p \text{ AND NOT}(s) \text{ AND } k$
- Given that an arbitrary signal $t=1$ for 5 seconds after key inserted. Turn on warning light when $t=1$ (to check that warning lights are working)
- $w = (p \text{ AND NOT}(s) \text{ AND } k) \text{ OR } t$



Some Gate-Based Circuit Drawing Conventions

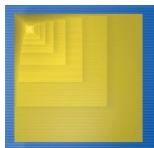


Boolean Algebra

- By defining logic gates based on Boolean algebra, we can use algebraic methods to manipulate circuits
 - So let's learn some Boolean algebraic methods
- Start with notation: Writing a AND b, a OR b, and NOT(a) is cumbersome
 - Use symbols: $a * b$, $a + b$, and a' (in fact, $a * b$ can be just ab).
 - Original: $w = (p \text{ AND NOT}(s) \text{ AND } k) \text{ OR } t$
 - New: $w = ps'k + t$
 - Spoken as “w equals p and s prime and k, or t”
 - Or even just “w equals p s prime k, or t”
 - s' known as “complement of s”
 - While symbols come from regular algebra, *don't* say “times” or “plus”

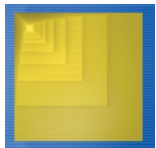
Boolean algebra precedence, highest precedence first.

Symbol	Name	Description
()	Parentheses	Evaluate expressions nested in parentheses first
'	NOT	Evaluate from left to right
*	AND	Evaluate from left to right
+	OR	Evaluate from left to right



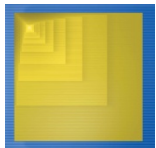
Boolean Algebra Operator Precedence

- Evaluate the following Boolean equations, assuming $a=1$, $b=1$, $c=0$, $d=1$.
 - Q1. $F = a * b + c$.
 - Answer: $*$ has precedence over $+$, so we evaluate the equation as $F = (1 * 1) + 0 = (1) + 0 = 1 + 0 = 1$.
 - Q2. $F = ab + c$.
 - Answer: the problem is identical to the previous problem, using the shorthand notation for $*$.
 - Q3. $F = ab'$.
 - Answer: we first evaluate b' because NOT has precedence over AND, resulting in $F = 1 * (1') = 1 * (0) = 1 * 0 = 0$.
 - Q4. $F = (ac)'$.
 - Answer: we first evaluate what is inside the parentheses, then we NOT the result, yielding $(1*0)' = (0)' = 0' = 1$.
 - Q5. $F = (a + b') * c + d'$.
 - Answer: Inside left parentheses: $(1 + (1')) = (1 + (0)) = (1 + 0) = 1$. Next, $*$ has precedence over $+$, yielding $(1 * 0) + 1' = (0) + 1' = 0 + 1' = 0 + 0 = 0$. The NOT has precedence over the OR, giving $(0) + (1') = (0) + (0) = 0 + 0 = 0$.



Boolean Algebra Terminology

- Example equation: $F(a,b,c) = a'bc + abc' + ab + c$
- **Variable**
 - Represents a value (0 or 1)
 - Three variables: a, b, and c
- **Literal**
 - Appearance of a variable, in true or complemented form
 - Nine literals: a' , b, c, a, b' , c' , a, b, and c
- **Product term**
 - Product of literals
 - Four product terms: $a'bc$, abc' , ab, c
- **Sum-of-products**
 - Equation written as OR of product terms only
 - Above equation is in sum-of-products form. “ $F = (a+b)c + d$ ” is not.
 - $F = (a')' + b$ is not. $F = (ab + bc)'$ is not.



Boolean Algebra Properties

- Commutative
 - $a + b = b + a$
 - $a * b = b * a$
- Distributive
 - $a * (b + c) = a * b + a * c$
 - $a + (b * c) = (a + b) * (a + c)$
 - (this one is tricky!)
- Associative
 - $(a + b) + c = a + (b + c)$
 - $(a * b) * c = a * (b * c)$
- Identity
 - $0 + a = a + 0 = a$
 - $1 * a = a * 1 = a$
- Complement
 - $a + a' = 1$
 - $a * a' = 0$
- To prove, evaluate all possibilities or use Axioms and theorems.

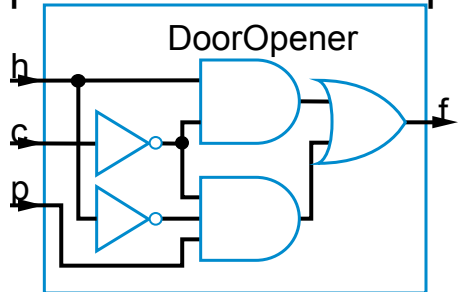
Example uses of the properties

- Show abc' equivalent to $c'ba$.
 - Use commutative property:
 - $a*b*c' = a*c'*b = c'*a*b = c'*b*a = c'ba$.
- Show $abc + abc' = ab$.
 - Use first distributive property
 - $abc + abc' = ab(c+c')$.
 - Complement property
 - Replace $c+c'$ by 1: $ab(c+c') = ab(1)$.
 - Identity property
 - $ab(1) = ab*1 = ab$.
- Show $x + x'z$ equivalent to $x + z$.
 - Second distributive property
 - Replace $x+x'z$ by $(x+x')*(x+z)$.
 - Complement property
 - Replace $(x+x')$ by 1,
 - Identity property
 - replace $1*(x+z)$ by $x+z$.



Example that Applies Boolean Algebra Properties

- Want automatic door opener circuit (e.g., for grocery store)
 - Output: $f=1$ opens door
 - Inputs:
 - $p=1$: person detected
 - $h=1$: door should be manually held open
 - $c=1$: door should stay closed
 - Want open door when
 - $h=1$ and $c=0$, or
 - $h=0$ and $p=1$ and $c=0$
 - Equation: $f = hc' + h'pc'$



- Found inexpensive chip that computes:

- $f = c'hp + c'hp' + c'h'p$

- Can we use it?

- Is it the same as $f = hc' + h'pc'$?

- Use Boolean algebra:

$$f = c'hp + c'hp' + c'h'p$$

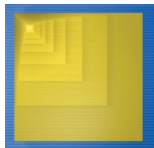
$$f = c'h(p + p') + c'h'p \quad (\text{by the distributive property})$$

$$f = c'h(1) + c'h'p \quad (\text{by the complement property})$$

$$f = c'h + c'h'p \quad (\text{by the identity property})$$

$$f = hc' + h'pc' \quad (\text{by the commutative property})$$

Same!



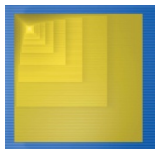
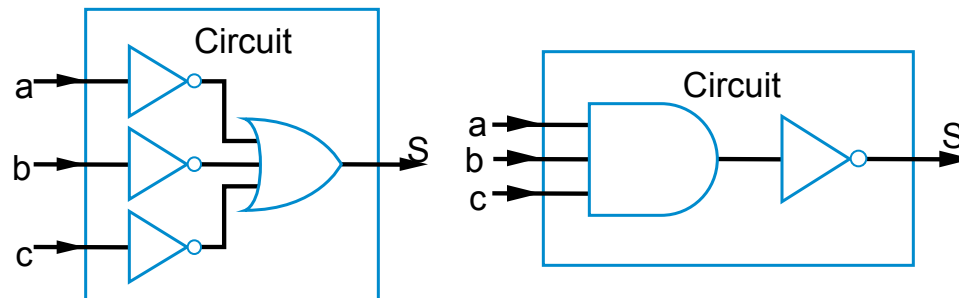
Boolean Algebra: Additional Properties

Aircraft lavatory sign example

- Null elements
 - $a + 1 = 1$
 - $a * 0 = 0$
- Idempotent Law
 - $a + a = a$
 - $a * a = a$
- Involution Law
 - $(a')' = a$
- DeMorgan's Law
 - $(a + b)' = a' b'$
 - $(ab)' = a' + b'$
 - Very useful!
- To prove, evaluate all possibilities or use Axioms and theorems.

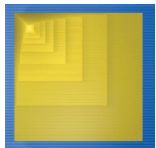
- Behavior
 - Three lavatories, each with sensor (a, b, c), equals 1 if door locked
 - Light “Available” sign (S) if any lavatory available
- Equation and circuit
 - $S = a' + b' + c'$
- Transform
 - $(abc)' = a' + b' + c'$ (by DeMorgan's Law)
 - $S = (abc)'$
- New equation and circuit

- Alternative: Instead of lighting “Available,” light “Occupied”
 - Opposite of “Available” function $S = a' + b' + c'$
 - So $S' = (a' + b' + c')'$
 - $S' = (a')' * (b')' * (c')'$ (by DeMorgan's Law)
 - $S' = a * b * c$ (by Involution Law)
 - Makes intuitive sense
 - Occupied if all doors are locked

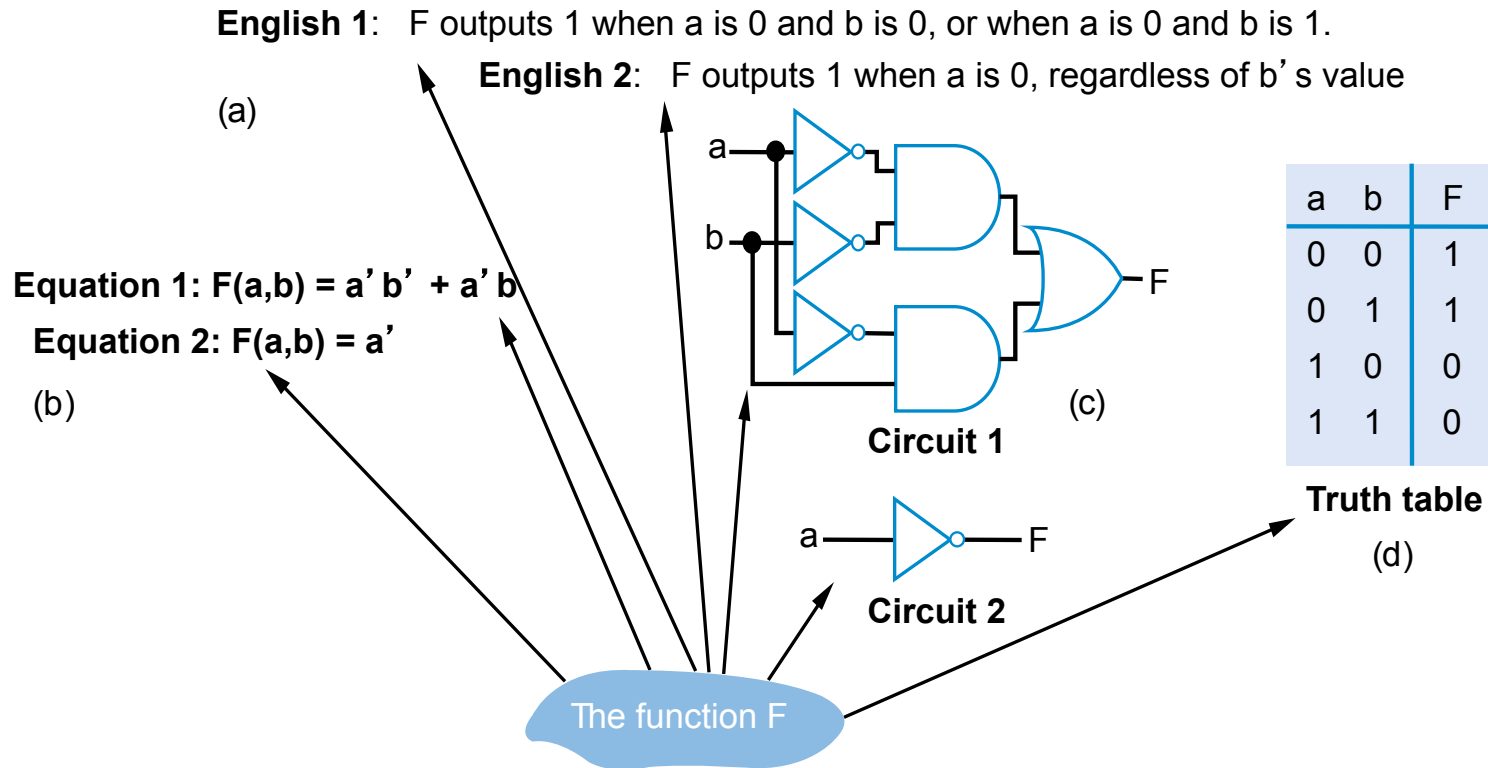


Exercises

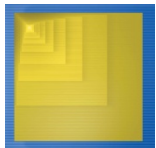
- Example 2.14
- Convert $F = ab(c+d)$ into SOP form. (hint: use distributivity)
- Convert $F = wx(x'y + zy' + xy)$ into SOP and make simplifications. Ans: $wxzy' + wxy$
- Prove that $x(x' + y(x' + y'))$ can never evaluate to 1.



Representations of Boolean Functions



- A function can be represented in different ways
 - Above shows seven representations of the same functions $F(a,b)$, using four different methods: English, Equation, Circuit, and Truth Table



Truth Table Representation of Boolean Functions

- Define value of function F for each possible combination of input values
 - 2-input function: 4 rows
 - 3-input function: 8 rows
 - 4-input function: 16 rows
- Q: Use truth table to define function $F(a,b,c)$ that is 1 when abc is 5 or greater in binary

a	b	F
0	0	
0	1	
1	0	
1	1	

(a)

a	b	c	F
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

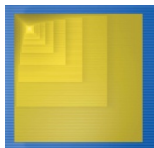
(b)

a	b	c	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

a

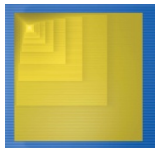
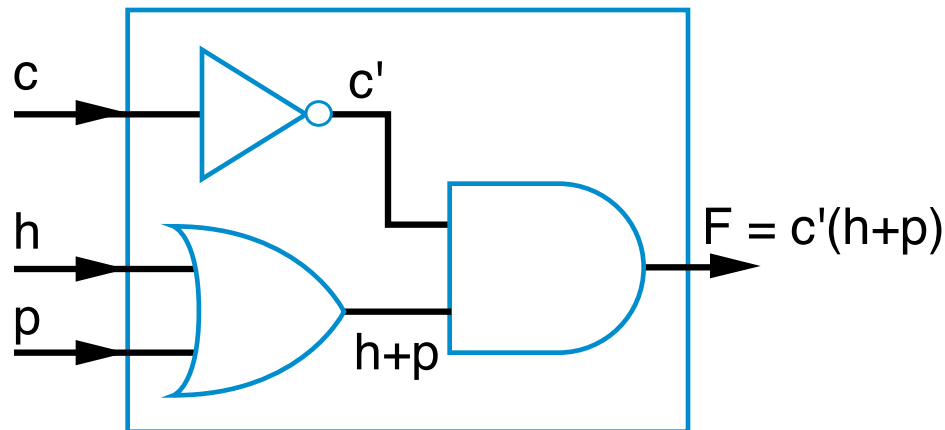
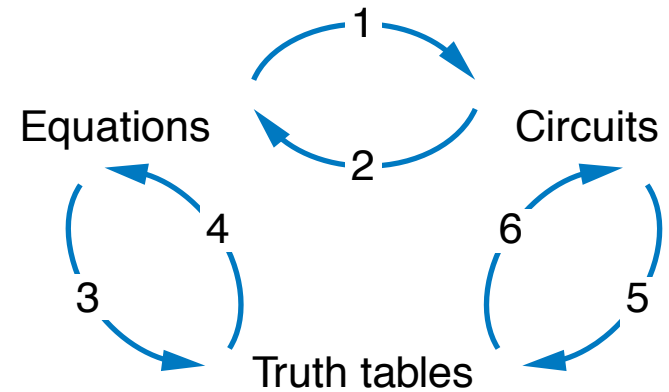
a	b	c	d	F
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

(c)

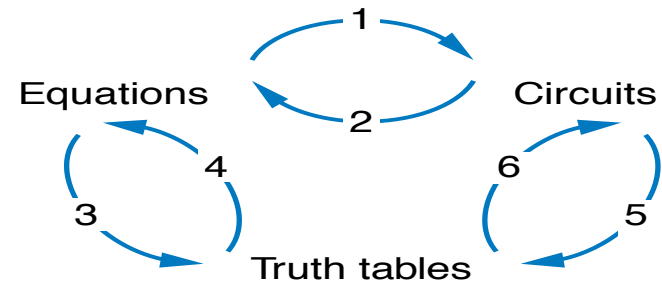


Converting among Representations

- Can convert from any representation to another
- Common conversions
 - Equation to circuit (we did this earlier)
 - Circuit to equation
 - Start at inputs, write expression of each gate output



Converting among Representations



- More common conversions
 - Truth table to equation (which we can then convert to circuit)
 - Easy—just OR each input term that should output 1
 - Equation to truth table
 - Easy—just evaluate equation for each input combination (row)
 - Creating intermediate columns helps

Inputs		Outputs	Term
a	b	F	F = sum of
0	0	1	$a' b'$
0	1	1	$a' b$
1	0	0	
1	1	0	

$$F = a' b' + a' b$$

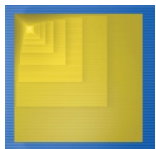
Q: Convert to equation

a	b	c	F	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	$ab' c$
1	1	0	1	abc'
1	1	1	1	abc

$$F = ab' c + abc' + abc$$

Q: Convert to truth table: $F = a' b' + a' b$

Inputs				Output
a	b	$a' b'$	$a' b$	F
0	0	1	0	1
0	1	0	1	1
1	0	0	0	0
1	1	0	0	0



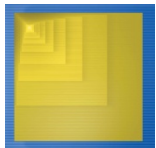
Example: Converting from Truth Table to Equation

- Parity bit: Extra bit added to data, intended to enable detection of error (a bit changed unintentionally)
 - e.g., errors can occur on wires due to electrical interference
- Even parity: Set parity bit so total number of 1s (data + parity) is even
 - e.g., if data is 001, parity bit is 1
→ 0011 has even number of 1s
- Want equation, but easiest to start from truth table for this example

a	b	c	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

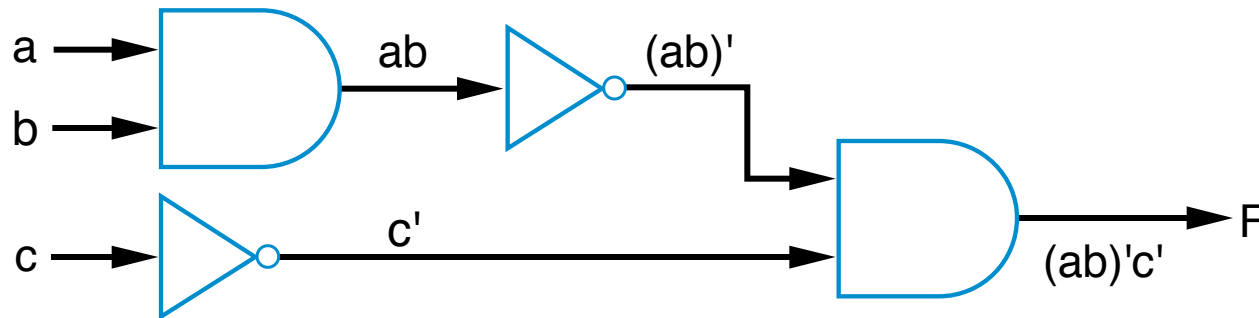
Convert to eqn.

$$P = a'b'c + a'bc' + ab'c' + abc$$

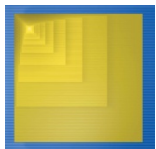


Example: Converting from Circuit to Truth Table

- First convert to circuit to equation, then equation to table



Inputs						Outputs
a	b	c	ab	$(ab)'$	c'	F
0	0	0	0	1	1	1
0	0	1	0	1	0	0
0	1	0	0	1	1	1
0	1	1	0	1	0	0
1	0	0	0	1	1	1
1	0	1	0	1	0	0
1	1	0	1	0	1	0
1	1	1	1	0	0	0



Exercise

- Example 2.20

